

Hands-on Activity 6.1

Strings and String Methods

Course Code: CPE-009A

Program: Computer Engineering

Course Title: Object Oriented Programming 1

Date Performed: 2/18/26

Section: CPE 12S1

Date Submitted: 2/21/26

Name(s): Ian Paulo S. Lisud

Instructor: Engr. Jimlord M. Quejado

6. Output

```
1 class Character():
2     def __init__(self, username):
3         self.__username = username
4         self.__hp = 100
5         self.__mana = 100
6         self.__damage = 5
7         self.__str = 0
8         self.__vit = 0
9         self.__int = 0
10        self.__agi = 0
11    def getUsername(self):
12        return self.__username
13    def setUsername(self, new_username):
14        self.__username = new_username
15
16    def getHp(self):
17        return self.__hp
18    def setHp(self, new_hp):
19        self.__hp = new_hp
20
21    def getDamage(self):
22        return self.__damage
23    def setDamage(self, new_damage):
24        self.__damage = new_damage
25
26    def getStr(self):
27        return self.__str
28    def setStr(self, new_str):
29        self.__str = new_str
30
31    def getVit(self):
32        return self.__vit
33    def setVit(self, new_vit):
34        self.__vit = new_vit
35
36    def getInt(self):
37        return self.__int
38    def setInt(self, new_int):
39        self.__int = new_int
40
41    def getAgi(self):
42        return self.__agi
43    def setAgi(self, new_agi):
44        self.__agi = new_agi
45
46    def reduceHp(self, damage_amount):
47        self.__hp = self.__hp - damage_amount
48    def addHp(self, heal_amount):
49        self.__hp = self.__hp + heal_amount
50
51    character1 = Character("Blee")
52    print (character1.getUsername())
```

non.uedu
Blee

Character.py

What was wrong with the initial command to print character 1 was the command `print(character1.__username)` will trigger an Attribute error because of the underscore. Due to the underscore python has made the variable private meaning you can't directly access character1,

```

1  from Character import Character
2
3  class Novice(Character):
4      def basicAttack(self, character):
5          character.reduceHp(self.getDamage())
6          print(f"{self.getUsername()} performed Basic Attack! -{self.getDamage()}")
7
8  character1 = Novice("Blee")
9  print(character1.getUsername())
10 print(character1.getHp())

```

```

Blee
Blee
100

```

Novice.py

Novice.py illustrates an implementation of Class Inheritance in Python, where the Novice class extends the functionality of a base Character class. By inheriting from Character, the Novice instance (character1) automatically gains access to attributes like hp and username without needing to redefine them. The output confirms that the initialized name "Blee" and set a default health value of 100.

For the test you need the following python files:

```

1  from Novice import Novice
2
3  class Swordsman(Novice):
4      def __init__(self, username):
5          super().__init__(username)
6          self.setStr(5)
7          self.setVit(10)
8          self.setHp(self.getHp()+self.getVit())
9
10     def slashAttack(self, character):
11         self.new_damage = self.getDamage() + self.getStr()
12         character.reduceHp(self.new_damage)
13         print(f"{self.getUsername()} performed Slash Attack! -{self.new_damage}")

```

Swordsman.py

```

1  from Novice import Novice
2  import random
3
4  class Archer(Novice):
5      def __init__(self, username):
6          super().__init__(username)
7          self.setAgi(5)
8          self.setStr(5)
9          self.setVit(5)
10         self.setHp(self.getHp()+self.getVit())
11
12     def rangeAttack(self, character):
13         self.new_damage = self.getDamage()+random.randint(0,self.getInt())
14         character.reduceHp(self.new_damage)
15         print(f"{self.getUsername()} perforemed Range Attack! -{self.new_damage}")
16

```

Archer.py

```

1  from Novice import Novice
2
3  class Mage(Novice):
4      def __init__(self, username):
5          super().__init__(username)
6          self.setInt(10)
7          self.setVit(5)
8          self.setHp(self.getHp()+self.getVit())
9
10     def heal(self):
11         self.addHp(self.getInt())
12         print(f"{self.getUsername()} performed Heal! +{self.getInt()}")
13
14     def magicAttack(self, character):
15         self.new_damage = self.getDamage()+self.getInt()
16         character.reduceHp(self.new_damage)
17         print(f"{self.getUsername()} perforemed Magic Attack! -{self.new_damage}")

```

Mage.py

```

1  from Swordsman import Swordsman
2  from Archer import Archer
3  from Mage import Mage
4
5  Character1 = Swordsman("Royce")
6  Character2 = Mage("Archie")
7
8  print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
9  print(f"{Character2.getUsername()} HP: {Character2.getHp()}")
10 Character1.slashAttack(Character2)
11 Character1.basicAttack(Character2)
12 print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
13 print(f"{Character2.getUsername()} HP: {Character2.getHp()}")
14 Character2.heal()
15 Character2.magicAttack(Character1)
16 print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
17 print(f"{Character2.getUsername()} HP: {Character2.getHp()}")

```

```

Blee
Blee
100
Royce HP: 110
Archie HP: 105
Royce performed Slash Attack! -10
Royce performed Basic Attack! -5
Royce HP: 110
Archie HP: 90
Archie performed Heal! +10
Archie performed Magic Attack! -15
Royce HP: 95
Archie HP: 100

```

Test.py

In the test while both characters likely inherit basic properties like hp and username from a parent Character class, the Swordsman and Mage classes have been extended with unique logic. For instance, the heal() method is exclusive to the Mage, adding a strategic layer to the combat that the basic Novice class didn't have.

```

1  from Swordsman import Swordsman
2  from Archer import Archer
3  from Mage import Mage
4
5  class Boss(Swordsman, Archer, Mage):
6      def __init__(self, username):
7          super().__init__(username)
8          self.setStr(10)
9          self.setVit(25)
10         self.setInt(5)
11         self.setHp(self.getHp()+self.getVit())
12

```

Boss.py

```

1 from Swordsman import Swordsman
2 from Archer import Archer
3 from Mage import Mage
4 from Boss import Boss
5
6 Character1 = Swordsman("Royce")
7 Character2 = Boss("Archie")
8
9 print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
10 print(f"{Character2.getUsername()} HP: {Character2.getHp()}")
11 Character1.slashAttack(Character2)
12 Character1.basicAttack(Character2)
13 print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
14 print(f"{Character2.getUsername()} HP: {Character2.getHp()}")
15 Character2.heal()
16 Character2.basicAttack(Character1)
17 Character2.slashAttack(Character1)
18 Character2.rangeAttack(Character1)
19 Character2.magicAttack(Character1)
20 print(f"{Character1.getUsername()} HP: {Character1.getHp()}")
21 print(f"{Character2.getUsername()} HP: {Character2.getHp()}")

```

```

Blee
Blee
100
Royce HP: 110
Archie HP: 145
Royce performed Slash Attack! -10
Royce performed Basic Attack! -5
Royce HP: 110
Archie HP: 130
Archie performed Heal! +5
Archie performed Basic Attack! -5
Archie performed Slash Attack! -15
Archie performed Range Attack! -7
Archie performed Magic Attack! -10
Royce HP: 73
Archie HP: 135

```

Test(Boss).py

By importing and using the Boss class, I have successfully created a "Super Class" that likely inherits from or mimics the abilities of the Swordsman, Archer, and Mage simultaneously. This is seen in Archie's ability to use slashAttack, rangeAttack, and magicAttack all in one sequence. This structure is excellent for game balance, as it allows you to create high-difficulty encounters by simply giving one object access to the entire library of methods you've already written. Additionally, the use of the Boss class shows a clear progression in your understanding of how to scale a project from a simple Novice to a complex, multi-layered encounter.

7. Supplementary Activity

```
1  import random
2  from Novice import Novice
3  from Swordsman import Swordsman
4  from Archer import Archer
5  from Mage import Mage
6  from Boss import Boss
7
8  class Game:
9      def __init__(self):
10         self.p1_wins = 0
11         self.p2_wins = 0
12
13     def select_role(self, name, allow_advanced=True):
14         if not allow_advanced:
15             return Novice(name)
16
17         print(f"\nSelect a role for {name}:")
18         print("1. Swordsman\n2. Archer\n3. Mage\n4. Novice")
19         choice = input("Enter choice (1-4): ")
20
21         if choice == "1": return Swordsman(name)
22         if choice == "2": return Archer(name)
23         if choice == "3": return Mage(name)
24         return Novice(name)
25
26     def combat_loop(self, player1, player2):
27         print(f"\n--- MATCH START: {player1.getUsername()} vs {player2.getUsername()} ---")
28
29         while player1.getHp() > 0 and player2.getHp() > 0:
30             # Randomize who goes first each turn
31             fighters = [player1, player2]
32             random.shuffle(fighters)
33
34             attacker = fighters[0]
35             defender = fighters[1]
36
37             print(f"\n{attacker.getUsername()}'s turn!")
38             # Basic logic: call the primary attack method of the class
39             if hasattr(attacker, 'slashAttack') and isinstance(attacker, Swordsman):
40                 attacker.slashAttack(defender)
41             elif hasattr(attacker, 'magicAttack') and isinstance(attacker, Mage):
42                 attacker.magicAttack(defender)
43             elif hasattr(attacker, 'rangeAttack') and isinstance(attacker, Archer):
44                 attacker.rangeAttack(defender)
```

```

45         else:
46             attacker.basicAttack(defender)
47
48         print(f"{defender.getUsername()} HP: {defender.getHp()}")
49
50         if defender.getHp() <= 0:
51             print(f"\n{defender.getUsername()} has been defeated!")
52             return attacker # Return the winner
53
54     def start(self):
55         while True:
56             print("\n==== GAME MENU =====")
57             print("1. Single Player (vs Boss)")
58             print("2. Player vs Player")
59             print("3. Exit")
60             mode = input("Select Mode: ")
61
62             if mode == "1":
63                 # Requirement: Start as Novice
64                 role_unlocked = self.p1_wins >= 2
65                 p1 = self.select_role("Player 1", allow_advanced=role_unlocked)
66                 boss = Boss("Monster")
67
68                 winner = self.combat_loop(p1, boss)
69                 if winner == p1:
70                     self.p1_wins += 1
71                     print(f"Total Wins: {self.p1_wins}")
72
73             elif mode == "2":
74                 p1 = self.select_role("Player 1")
75                 p2 = self.select_role("Player 2")
76
77                 winner = self.combat_loop(p1, p2)
78                 if winner == p1: self.p1_wins += 1
79                 else: self.p2_wins += 1
80                 print(f"Scores -> P1: {self.p1_wins} | P2: {self.p2_wins}")
81
82             elif mode == "3":
83                 break
84
85     if __name__ == "__main__":
86         my_game = Game()
87         my_game.start()

```

Result (Single Player vs Boss)

--- MATCH START: Player 1 vs Monster ---

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 140

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 135

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 130

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 85

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 125

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 70

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 55

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 120

Player 1's turn!

Player 1 performed Basic Attack! -5

Monster HP: 115

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 40

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 25

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: 10

Monster's turn!

Monster performed Slash Attack! -15

Player 1 HP: -5

Player 1 has been defeated!

Result (1v1)

--- MATCH START: Player 1 vs Player 2 ---

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 105

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 100

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 95

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 90

Player 1's turn!

Player 1 perforemed Slash Attack! -10

Player 2 HP: 95

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 85

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 80

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 75

Player 1's turn!

Player 1 perforemed Slash Attack! -10

Player 2 HP: 85

Player 2's turn!

Player 2 perforemed Range Attack! -5

Player 1 HP: 70

Player 1's turn!

Player 1 perforemed Slash Attack! -10

Player 2 HP: 75

Player 1's turn!

Player 1 perforemed Slash Attack! -10

Player 2 HP: 65

Player 1's turn!

Player 1 perforemed Slash Attack! -10

Player 2 HP: 55

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 65

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 60

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 55

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 50

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: 45

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: 35

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 45

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: 25

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: 15

Player 2's turn!
Player 2 performed Range Attack! -5
Player 1 HP: 40

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: 5

Player 1's turn!
Player 1 performed Slash Attack! -10
Player 2 HP: -5

Player 2 has been defeated!
Scores -> P1: 1 | P2: 0

In developing this Game.py script, I have successfully transitioned from writing isolated classes to architecting a fully functional, turn-based Game Engine. By integrating the Novice, Swordsman, Archer, Mage, and Boss classes, I've created a centralized system that manages complex object interactions and persistent game state. My implementation of the select_role method serves as a gateway for Dynamic Instantiation, where the player's progression is directly tied to their performance; by checking self.p1_wins, I've enforced a gameplay rule that restricts advanced roles until the player has proven their skill through at least two victories. Within the combat_loop, I utilized Python's random module to ensure that turn order is non-deterministic, adding a layer of unpredictability and fairness to the encounter. I also leveraged Introspection techniques—specifically hasattr() and isinstance()—to allow the engine to intelligently decide which attack method to call based on the specific type of character currently acting. This approach ensures that a Swordsman uses their slashAttack while a Novice defaults to a basicAttack, all within the same streamlined loop. By encapsulating the entire experience within the Game class

8. Conclusion

I'm undergoing a core progression of Object-Oriented Programming (OOP), moving from defining a single entity to architecting a dynamic, multi-layered system. By starting with the Novice class, you learned Inheritance (attributes and methods) of a parent Character without redundant coding. Adding the Swordsman, Archer, and Mage, which allowed different objects to perform unique actions like healing or magic attacks while interacting through a shared interface.